



UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Projeto de Software Web, Móvel e na Nuvem Sistema de Gestão Digital para a Oficina Duarte & Raposo

Tiago Dias Pereira - 55019

7 de junho de 2026

Conteúdo

Resumo	3
1 Introdução	4
1.1 Âmbito e Motivação	4
1.2 Objetivos da Implementação	4
2 Engenharia de Requisitos	5
2.1 Levantamento de Dados Físicos	5
2.2 Histórias de Utilizador (<i>User Stories</i>)	5
3 Modelação de Dados e Arquitetura	5
3.1 Modelo Relacional	5
3.2 A Escolha do Docker	6
4 Próximos Passos	6
5 Anexos	7
Anexos	7
Acrónimos	8

Resumo

Este relatório descreve a primeira fase de desenvolvimento do meu projeto final para a unidade curricular de Projeto de Software Web, Móvel e na Nuvem. O objetivo prático é resolver um problema com o qual lido diariamente no meu local de trabalho: a desorganização e a perda de tempo geradas pelo uso de folhas de papel na oficina Duarte & Raposo, no Canhoso.

A solução passa por criar um sistema de gestão desenhado à medida, com uma PWA (Progressive Web App) pensada para os mecânicos usarem no telemóvel, e um *Dashboard* em React para a gestão da oficina. Este documento apresenta o levantamento das necessidades feito diretamente na oficina, a estruturação da base de dados relacional em SQL Server, e a justificação técnica para a utilização de *containers* com o Docker.

Palavras-chave

React; Node.js; SQL Server; PWA; Docker; Gestão de Oficina; Autocarranas.

1 Introdução

1.1 Âmbito e Motivação

No dia a dia da oficina Duarte & Raposo, especializada em eletricidade, mecânica e autocaravanas, todo o processo de entrada de veículos e registo de peças é feito de forma manual. Este sistema baseado em papel tem várias falhas: é difícil consultar o histórico de uma autocaravana que já lá esteve no ano passado, os papéis sujam-se ou perdem-se pela oficina, e no final da reparação perde-se muito tempo a fazer contas à mão para conseguir passar a fatura ao cliente.

A motivação para este projeto surge exatamente da vontade de juntar o útil ao agradável: aplicar os conhecimentos adquiridos ao longo do curso de Informática Web para desenvolver uma solução real que facilite não só o meu trabalho, mas o de toda a equipa. O foco vai estar especialmente nas autocaravanas, dado que são veículos complexos (com parte mecânica e parte habitacional) e exigem um registo mais detalhado.

1.2 Objetivos da Implementação

- **Acabar com o papel:** Passar todo o registo de clientes, veículos e folhas de obra para uma base de dados digital.
- **Aplicação Móvel (PWA):** Desenvolver uma interface em **React** leve e direta, para que qualquer mecânico consiga dar entrada a um carro no sistema usando o telemóvel, sem perder muito tempo.
- **Gestão no Computador:** Criar uma interface web *desktop* para o Duarte e o Raposo poderem ver o que está a ser feito, consultar históricos e ver as contas automáticas do IVA para a faturação.
- **Arquitetura e Portabilidade:** Construir o *backend* em Node.js com base de dados Structured Query Language (SQL) Server, e utilizar o **Docker** para contentorizar o projeto. Isto vai garantir que o sistema corre sempre bem, quer esteja a programar no meu computador fixo ou num portátil novo, sem dores de cabeça com compatibilidades.

2 Engenharia de Requisitos

2.1 Levantamento de Dados Físicos

Como trabalho diretamente na oficina, o levantamento de requisitos foi muito direto. Peguei nas folhas de obra e faturas em branco que usamos atualmente e retirei a informação essencial que tem de transitar para a plataforma digital:

- **Dados do Cliente:** Nome, Morada, NIF e Telefone.
- **Dados do Veículo:** Matrícula, e, no caso das autocaravanas, a separação entre a Marca/Modelo do Chassis e a Marca da Célula.
- **Dados da Entrada:** Data, os quilómetros (KMS) com que o carro entrou, e espaço para observações.
- **O Serviço:** As várias linhas de material gasto e horas de trabalho, com quantidade, preço unitário e categoria do serviço.

2.2 Histórias de Utilizador (*User Stories*)

Com base nas rotinas da equipa, estruturei o que o sistema tem de fazer:

- *"Como mecânico, preciso de abrir a app no telemóvel e registar a entrada de um carro em poucos cliques, para voltar rapidamente ao trabalho."*
- *"Como mecânico, quero adicionar as peças e serviços que usei diretamente à folha de obra ativa daquele carro."*
- *"Como mecânico, preciso de uma secção de 'Conselhos' para avisar o cliente sobre peças que vão precisar de ser trocadas no futuro, para prestarmos um bom serviço preventivo."*
- *"Como gestor, preciso que o sistema some os valores do material e faça o cálculo do IVA sozinho quando a reparação acaba."*

3 Modelação de Dados e Arquitetura

3.1 Modelo Relacional

Para garantir a integridade referencial dos dados, mitigar redundâncias e assegurar a segurança e rastreabilidade das operações (*Auditing*), desenhei

uma base de dados relacional que cumpre com os normativos da Terceira Forma Normal (3NF). O esquema consolida-se agora em cinco entidades principais, incluindo um sistema de *Soft Delete* (campo ‘Ativo’) para preservação de histórico fiscal:

1. **Colaboradores:** ID_Colaborador, Nome, Cargo, Email, Password.Hash, Ativo.
2. **Clientes:** ID_Cliente, Nome, Contribuinte.NIF, Telefone, Morada, Ativo.
3. **Autocaravanas:** Matricula, Marca_Chassis, Modelo_Chassis, Marca_Celula, Ano, Ativo, *ID_Cliente (FK)*.
4. **Folhas_Obra:** ID_Folha, Data_Entrada, KMS_Entrada, Estado_Reparacao, Observacoes, Conselhos, *Matricula (FK)*, *ID_Colaborador (FK)*.
5. **Linhas_Reparacao:** ID_Linha, Designacao, Quantidade, Valor_Unitario, Categoria, *ID_Folha (FK)*.

Os diagramas visuais correspondentes, refletindo as regras de segurança e associação de colaboradores, encontram-se na secção de Anexos.

3.2 A Escolha do Docker

Em termos de arquitetura, decidi usar o Docker neste projeto. A vantagem principal para o meu caso de uso é a facilidade de criar ambientes isolados. Ao definir um ficheiro `docker-compose.yml`, consigo ter a base de dados SQL Server e a API em Node.js a correr em qualquer computador através de um único comando. Isto evita que o meu sistema operativo fique “sujo” com serviços em segundo plano e facilita muito a transição do projeto entre computadores de desenvolvimento e, futuramente, para produção.

4 Próximos Passos

Com a arquitetura desenhada e a base de dados estruturada, o próximo passo prático passa pela configuração do ambiente com o Docker, inicialização do projeto no GitHub, e começo do desenvolvimento da Application Programming Interface (API) em Node.js para ligar à base de dados.

5 Anexos

Anexo I: Diagrama da Base de Dados

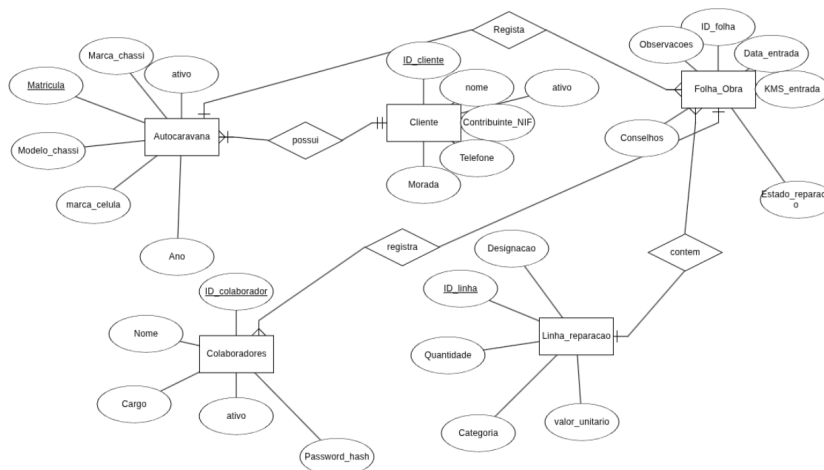


Figura 1: Diagrama concetual gerado no ERDPlus.

Anexo II: Esquema Relacional da Base de Dados

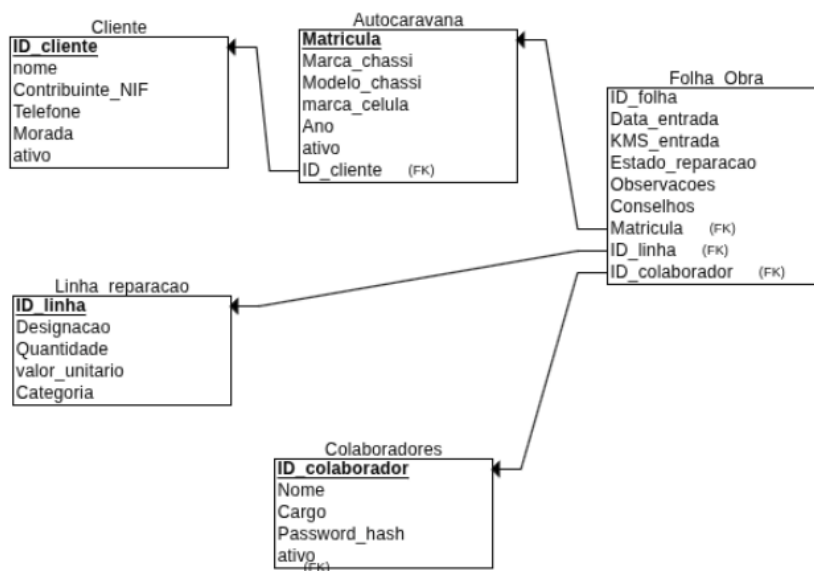


Figura 2: Esquema Relacional final gerado no ERDPlus.

Acrónimos

UBI Universidade da Beira Interior

API Application Programming Interface

SQL Structured Query Language

PWA Progressive Web App